

Engenharia de Serviços em Rede

Trabalho Prático N°1

Streaming de áudio e vídeo a pedido e em tempo real

MEI - 2023/2024

André Nunes
A85635



Afonso Bessa
PG53597



Francisco Claudino
PG50380



Grupo 66



Universidade do Minho

Conteúdo

1	Etapa 1 - Streaming HTTP simples sem adaptação dinâmica de débito	3
1.1	Topologia	3
1.2	Procedimento	4
1.3	Questões	7
2	Etapa 2 - Streaming adaptativo sobre HTTP (MPEG-DASH)	11
2.1	Procedimento	11
2.2	Questões	11
3	Etapa 3 - Streaming RTP/RTCP unicast sobre UDP e multicast com anúncios SAP	15
3.1	Topologia <i>Unicast</i>	15
3.2	Procedimento <i>Unicast</i>	16
3.3	Topologia <i>Multicast</i>	17
3.4	Procedimento <i>Multicast</i>	18
3.5	Questões	19

Lista de Figuras

1.1	Topologia da Rede	3
1.2	Ping teste do portátil Monstro para o servidor VStreamer	4
1.3	VStreamer a transmitir o videoA.mp4	4
1.4	VStreamer a transmitir o videoA.mp4 e portátil Jasmine a receber a stream	5
1.5	Tráfego registado no Wireshark da transmissão do VStreamer do videoA.mp4	5
1.6	Página HTML onde o VStreamer esta a transmitir o videoA.mp4	6
1.7	VStreamer esta a transmitir o videoA.mp4	6
1.8	Taxa Teórica de bps necessária dada pelo ffmpeg	7
1.9	Taxa Prática de bps necessária para transmitir para o portátil Jasmine	7
1.10	Taxa Prática de bps necessária para transmitir para o portátil Bela	7
1.11	Taxa Prática de bps necessária para transmitir para o portátil Monstro	8
1.12	Fluxos gerados	8
1.13	Número de Fluxos Gerados - Primeira Resolução	9
1.14	Número de Fluxos Gerados - Resolução Correta	9
1.15	Exemplo da Pilha Protocolar	9
1.16	Assumindo um aumento de utilização linear ao longo de um tempo t	9

2.1	Vídeos com resoluções diferentes e ficheiro de manifesto	11
2.2	VStreamer a servir conteúdo com o <i>minihttpd</i>	11
2.3	Limitação da largura de banda da ligação do cliente Bela para o vídeo "videoB 200 150 200k"	12
2.4	Limitação da largura de banda da ligação do cliente Bela para o vídeo "videoB 480 360 500k"	12
2.5	Limitação da largura de banda da ligação do cliente Bela para o vídeo "videoB 640 480 1000k"	12
2.6	Visualização da pilha protocolar numa captura	12
2.7	Streaming no cliente Alladin	13
2.8	Streaming no cliente Alladin	13
2.9	Captura do Wireshark do streaming no cliente Bela com limitação da largura de banda	14
2.10	Trama do streaming com débito adaptativo	14
3.1	Tipologia de Rede para o exercício <i>Unicast</i>	15
3.2	Ping teste do VStreamer para o portátil Monstro	16
3.3	VStreamer a transmitir para o portátil Monstro	16
3.4	Tipologia de Rede para o exercício <i>Multicast</i>	17
3.5	Ping teste do VStreamer para todos os portáteis da topologia	18
3.6	VStreamer a transmitir para todos os portáteis da rede	18
3.7	Cenário <i>Unicast</i>	19
3.8	Cenário <i>Multicast</i>	19

Capítulo 1

Etapa 1 - Streaming HTTP simples sem adaptação dinâmica de débito

1.1 Topologia

Tal como requisitado, a topologia utilizada foi a seguinte:

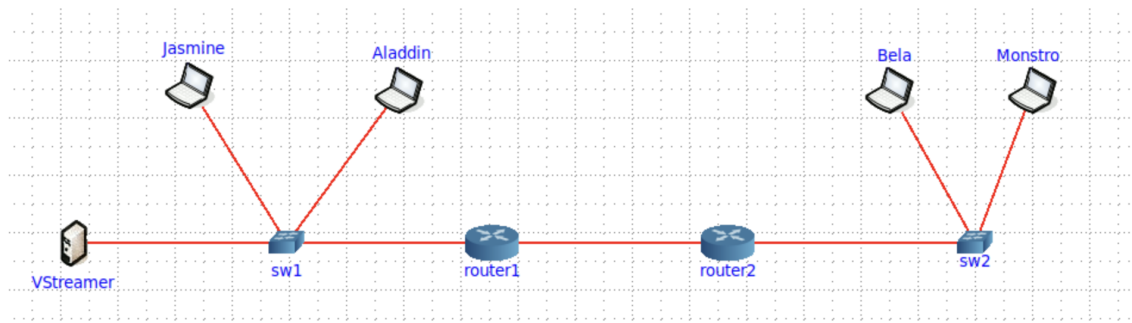


Figura 1.1: Topologia da Rede

1.2 Procedimento

Antes de dar início ao procedimento, verificamos que a conexão estava funcional efetuando um *ping* do portátil Monstro para o servidor VStreamer, tal como é possível visualizar pela figura 1.2:

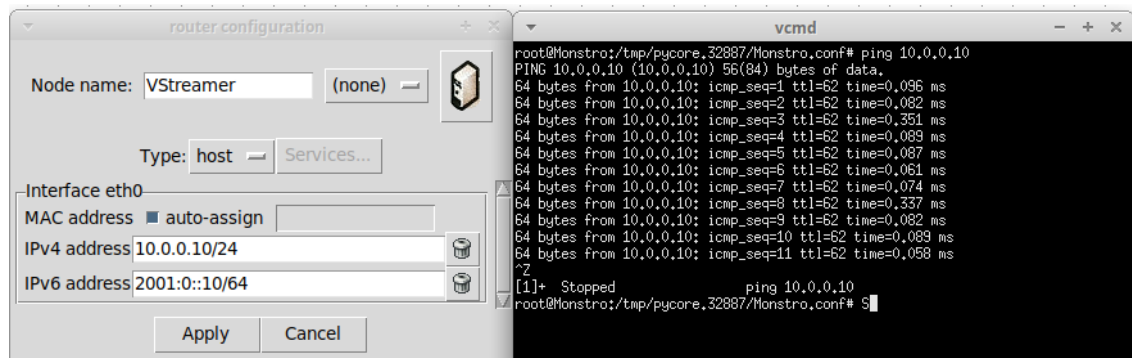


Figura 1.2: Ping teste do portátil Monstro para o servidor VStreamer

O procedimento utilizado foi exatamente como o requisitado:

1. No servidor VStreamer colocamos o VLC a fazer stream do videoA.mp4:

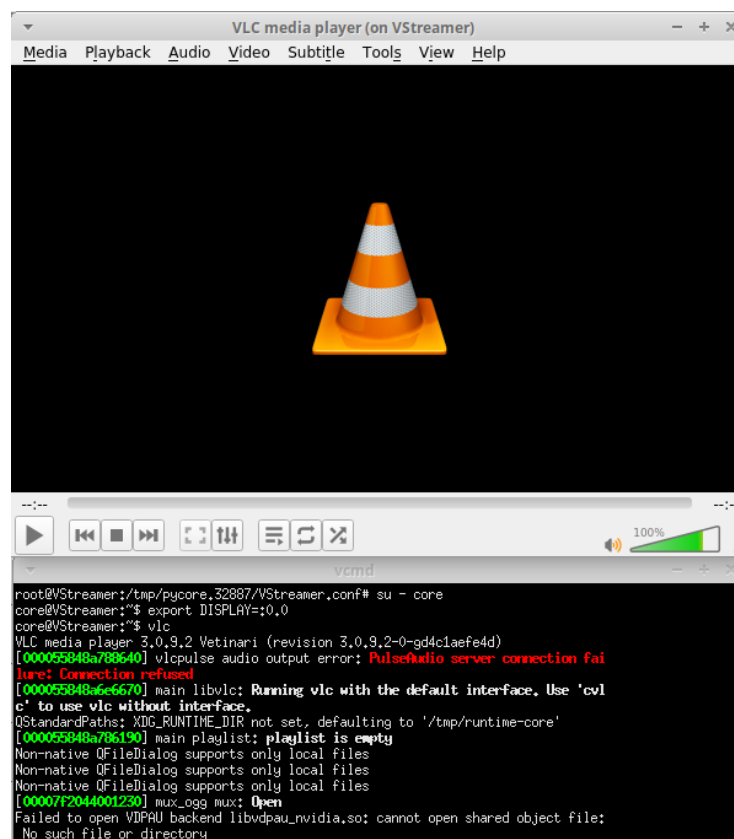


Figura 1.3: VStreamer a transmitir o videoA.mp4

- De seguida, colocamos o VLC do portátil Jasmine a receber a *stream*, como se pode observar na próxima figura, registrando também o tráfego no Wireshark:

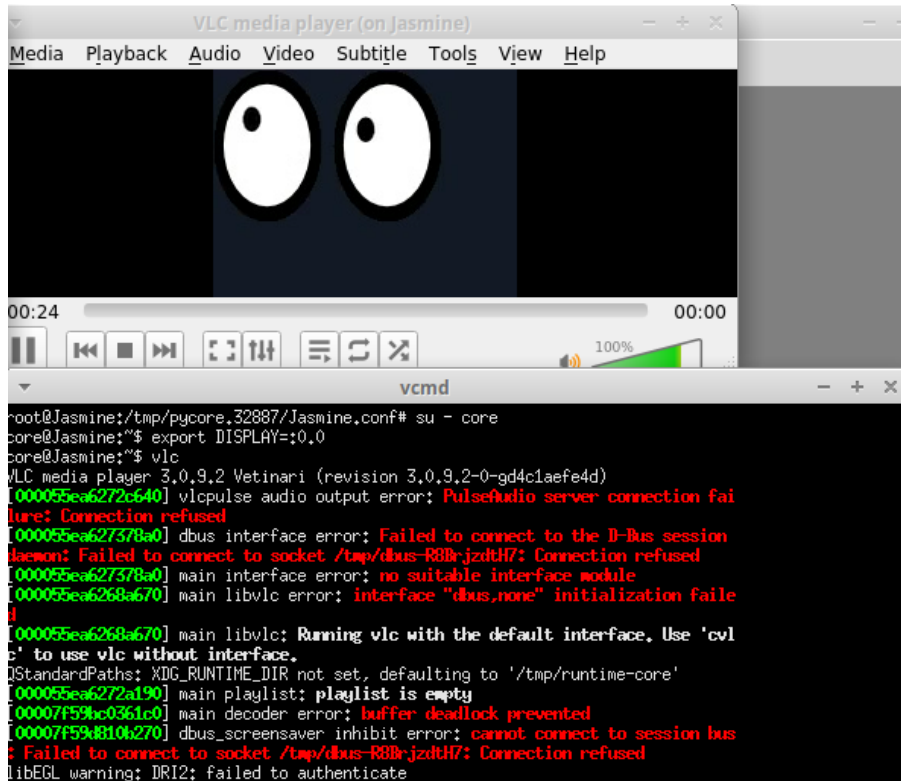


Figura 1.4: VStreamer a transmitir o videoA.mp4 e portátil Jasmine a receber a stream

No.	Time	Source	Destination	Protocol	Length	Info
108	178.457945121	10.0.0.1	224.0.0.5	OSPF	78	Hello Packet
109	180.469041442	10.0.0.1	224.0.0.5	OSPF	78	Hello Packet
110	180.497421630	10.0.0.20	10.0.0.10	TCP	78	58782 → 8080 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1
111	180.497437936	10.0.0.10	10.0.0.20	TCP	74	8080 → 58782 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460
112	180.497447992	10.0.0.20	10.0.0.10	TCP	66	58782 → 8080 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=27771971
113	180.497490561	10.0.0.20	10.0.0.10	HTTP	200	GET / HTTP/1.1
114	180.497493346	10.0.0.10	10.0.0.20	TCP	66	8080 → 58782 [ACK] Seq=1 Ack=135 Win=65152 Len=0 TSval=390655
115	180.525139205	fe80::200:ff:feaa:3	ff02::5	OSPF	90	Hello Packet
116	180.525203639	10.0.0.10	10.0.0.20	TCP	169	8080 → 58782 [PSH, ACK] Seq=1 Ack=135 Win=65152 Len=103 TSval=2777
117	180.525220366	10.0.0.20	10.0.0.10	TCP	66	58782 → 8080 [ACK] Seq=135 Ack=104 Win=64256 Len=0 TSval=2777
118	180.525233347	10.0.0.10	10.0.0.20	TCP	1514	8080 → 58782 [ACK] Seq=104 Ack=135 Win=65152 Len=1448 TSval=2777
119	180.525233347	10.0.0.10	10.0.0.20	TCP	1514	8080 → 58782 [ACK] Seq=1552 Ack=135 Win=65152 Len=1448 TSval=2777
120	180.525233507	10.0.0.10	10.0.0.20	TCP	543	8080 → 58782 [PSH, ACK] Seq=3000 Ack=135 Win=65152 Len=477 TSval=2777
121	180.525240408	10.0.0.20	10.0.0.10	TCP	66	58782 → 8080 [ACK] Seq=135 Ack=1552 Win=64128 Len=0 TSval=2777
122	180.525240869	10.0.0.20	10.0.0.10	TCP	66	58782 → 8080 [ACK] Seq=135 Ack=3000 Win=63488 Len=0 TSval=2777
123	180.525241270	10.0.0.20	10.0.0.10	TCP	66	58782 → 8080 [ACK] Seq=135 Ack=3477 Win=63232 Len=0 TSval=2777
124	180.525250074	10.0.0.10	10.0.0.20	TCP	1514	8080 → 58782 [ACK] Seq=3477 Ack=135 Win=65152 Len=1448 TSval=2777
125	180.525250164	10.0.0.10	10.0.0.20	TCP	1514	8080 → 58782 [ACK] Seq=4925 Ack=135 Win=65152 Len=1448 TSval=2777
126	180.525250234	10.0.0.10	10.0.0.20	TCP	1514	8080 → 58782 [ACK] Seq=6373 Ack=135 Win=65152 Len=1448 TSval=2777
127	180.525250294	10.0.0.10	10.0.0.20	TCP	672	8080 → 58782 [PSH, ACK] Seq=7821 Ack=135 Win=65152 Len=606 TSval=2777
128	180.525258467	10.0.0.20	10.0.0.10	TCP	66	58782 → 8080 [ACK] Seq=135 Ack=4025 Win=62464 Len=0 TSval=2777
129	180.525258888	10.0.0.20	10.0.0.10	TCP	66	58782 → 8080 [ACK] Seq=135 Ack=6373 Win=61824 Len=0 TSval=2777
130	180.525259269	10.0.0.20	10.0.0.10	TCP	66	58782 → 8080 [ACK] Seq=135 Ack=7821 Win=61056 Len=0 TSval=2777
131	180.525259649	10.0.0.20	10.0.0.10	TCP	66	58782 → 8080 [ACK] Seq=135 Ack=8427 Win=60800 Len=0 TSval=2777
132	180.685147888	10.0.0.10	10.0.0.20	TCP	1514	8080 → 58782 [ACK] Seq=8427 Ack=135 Win=65152 Len=1448 TSval=2777

Figura 1.5: Tráfego registado no Wireshark da transmissão do VStreamer do videoA.mp4

3. De seguida, colocamos a página HTML dada numa instância do Firefox no portátil Bela:



Figura 1.6: Página HTML onde o VStreamer esta a transmitir o videoA.mp4

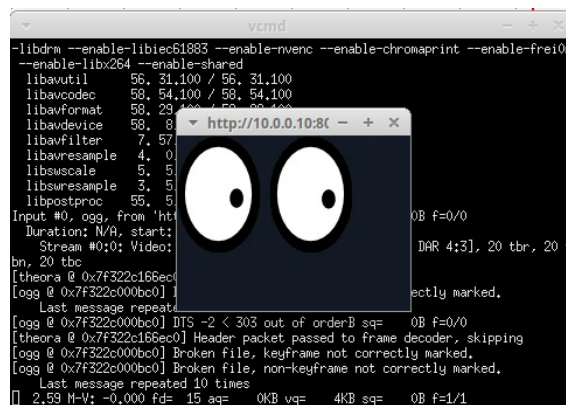


Figura 1.7: VStreamer esta a transmitir o videoA.mp4

1.3 Questões

Questão 1: Capture três pequenas amostras de tráfego no link de saída do servidor, respectivamente com 1 cliente (VLC), com 2 clientes (VLC e Firefox) e com 3 clientes (VLC, Firefox e ffplay). Identifique a taxa em bps necessária (usando o *ffmpeg -i videoA.mp4* e/ou o próprio wireshark), o encapsulamento usado e o número total de fluxos gerados. Comente a escalabilidade da solução. Ilustre com evidências da realização prática do exercício (ex: capturas de ecrã).

Taxa Teórica: Como é possível observar na imagem seguinte, a taxa teórica em bps necessária para a *stream* do nosso *videoA.mp4* é de 10 kb/s.

```
vcmd
-lbdrm --enable-libiec61883 --enable-nvenc --enable-chromaprint --enable-frei0r
--enable-libx264 --enable-shared
libavutil      56. 31.100 / 56. 31.100
libavcodec     58. 54.100 / 58. 54.100
libavformat    58. 29.100 / 58. 29.100
libavdevice    58.  8.100 / 58.  8.100
libavfilter     7. 57.100 /  7. 57.100
libavresample   4.  0.  0 /  4.  0.  0
libswscale     5.  5.100 /  5.  5.100
libswresample   3.  5.100 /  3.  5.100
libpostproc    55.  5.100 / 55.  5.100
Input #0, mov,mp4,m4a,3gp,3g2,mj2, from 'videoA.mp4':
Metadata:
  major_brand      : isom
  minor_version    : 512
  compatible_brands: isomiso2avc1mp41
  encoder         : Lavf58.29.100
Duration: 00:00:15.20, start: 0.000000, bitrate: 13 kb/s
Stream #0:0(und): Video: h264 (High) (avc1 / 0x31637661), yuv420p, 200x150,
10 kb/s, 20 fps, 20 tbr, 10240 tbn, 40 tbc (default)
Metadata:
  handler_name     : VideoHandler
At least one output file must be specified
core@VStream:~$
```

Figura 1.8: Taxa Teórica de bps necessária dada pelo ffmpeg

Taxa Prática: As taxas práticas que estávamos à espera de encontrar eram valores acima do valor teórico (devido a retransmissões e perdas de pacotes) e que fossem semelhantes entre si.

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s
Frame	100.0	192	100.0	87234	11 k	0	0	0
Ethernet	100.0	192	3.1	2688	344	0	0	0
Internet Protocol Version 6	16.7	32	1.5	1280	163	0	0	0
Internet Protocol Version 4	82.3	158	3.6	3168	405	0	0	0
Transmission Control Protocol	65.1	125	89.0	77678	9944	124	77512	9923
Hypertext Transfer Protocol	0.5	1	0.2	134	17	1	134	17
Open Shortest Path First	16.1	31	1.6	1364	174	31	1364	174
Internet Group Management Protocol	1.0	2	0.0	32	4	2	32	4
Address Resolution Protocol	1.0	2	0.1	56	7	2	56	7

Figura 1.9: Taxa Prática de bps necessária para transmitir para o portátil Jasmine

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s
Frame	100.0	2714	100.0	1739350	77 k	0	0	0
Ethernet	100.0	2714	2.2	37996	3695	0	0	0
Internet Protocol Version 6	1.9	51	0.1	2040	91	0	0	0
Internet Protocol Version 4	97.6	2649	3.0	52988	2364	0	0	0
Transmission Control Protocol	94.2	2557	94.3	1640356	73 k	2554	1639669	73 k
Hypertext Transfer Protocol	0.1	3	0.0	591	26	3	591	26
Open Shortest Path First	3.3	90	0.2	3960	176	90	3960	176
Internet Group Management Protocol	0.1	2	0.0	32	1	2	32	1
Address Resolution Protocol	0.5	14	0.0	392	17	14	392	17

Figura 1.10: Taxa Prática de bps necessária para transmitir para o portátil Bela

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s
Frame	100.0	1284	100.0	792136	50 k	0	0	0
Ethernet	100.0	1284	2.3	17976	1143	0	0	0
Internet Protocol Version 6	3.5	45	0.2	1800	114	0	0	0
Internet Protocol Version 4	95.7	1229	3.1	24588	1563	0	0	0
Transmission Control Protocol	90.7	1164	93.8	743298	47 k	1162	742777	47 k
Hypertext Transfer Protocol	0.2	2	0.1	457	29	2	457	29
Open Shortest Path First	4.9	63	0.3	2772	176	63	2772	176
Internet Group Management Protocol	0.2	2	0.0	32	2	2	32	2
Address Resolution Protocol	0.8	10	0.0	280	17	10	280	17

Figura 1.11: Taxa Prática de bps necessária para transmitir para o portátil Monstro

O que vemos são valores semelhantes em duas das capturas, mas em todas elas verificam-se as taxas esperadas, as quais correspondem a valores superiores aos teóricos (15 a 30 kb/s).

Número de fluxos gerados: Na nossa primeira execução, vimos que houve dois fluxos gerados. Verificamos que dois deles tinham o mesmo IP e portas diferentes, o que pode ser explicado por um *refresh* na página HTML do Firefox. Devido a isto, para obtermos um cálculo dos bps mais acertado, refizemos a captura do tráfego para o passo do Firefox, o que permitiu obter o número correto de fluxos gerados, três.

1. VStreamer → Jasmine
2. VStreamer → Bela
3. VStreamer → Monstro

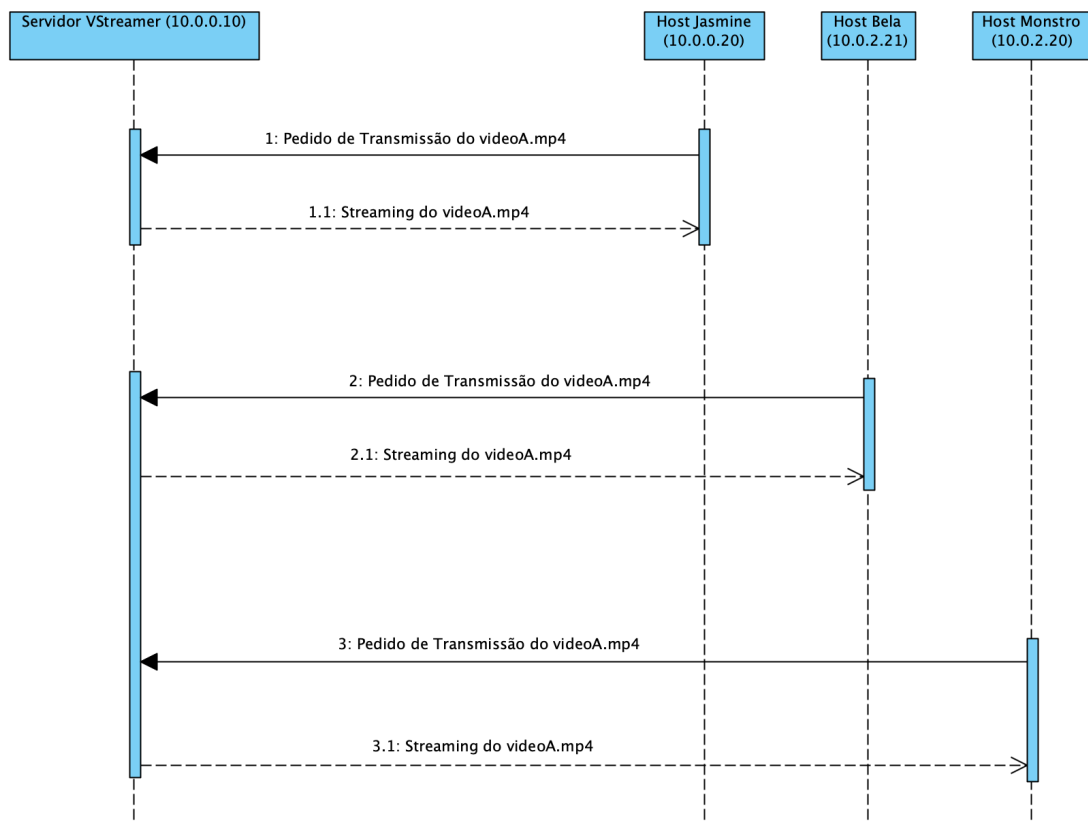


Figura 1.12: Fluxos gerados

Address A	Port A	Address B	Port B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A
10.0.0.20	42040	10.0.0.10	8080	1,960	1332 k	980	64 k	980	1268 k	0.000000	151.7517	3409	
10.0.2.20	55736	10.0.0.10	8080	911	616 k	456	30 k	455	585 k	82.965028	68.7868	3516	
10.0.2.21	44988	10.0.0.10	8080	1,061	716 k	531	35 k	530	681 k	59.992423	79.5831	3550	
10.0.2.21	54788	10.0.0.10	8080	199	131 k	100	6936	99	124 k	139.431327	12.3205	4503	

Figura 1.13: Número de Fluxos Gerados - Primeira Resolução

Address A	Port A	Address B	Port B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A
10.0.0.20	42040	10.0.0.10	8080	1,244	847 k	622	41 k	622	806 k	0.000000	94.6736	3468	
10.0.2.20	55736	10.0.0.10	8080	195	130 k	98	6610	97	123 k	82.965028	11.7092	4516	
10.0.2.21	44988	10.0.0.10	8080	513	345 k	257	17 k	256	327 k	59.992423	34.6813	3989	

Figura 1.14: Número de Fluxos Gerados - Resolução Correta

Encapsulamento utilizado: O encapsulamento utilizado pode ser identificado por meio da análise das capturas no Wireshark. É evidente que a pilha de protocolos em uso é a seguinte:

HTTP → TCP → IP → Ethernet

conforme demonstrado na figura a seguir:

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s
Frame	100.0	1274	100.0	810431	60 k	0	0	0
Ethernet	100.0	1274	2.2	17836	1340	0	0	0
Internet Protocol Version 6	2.0	25	0.1	1000	75	0	0	0
Internet Protocol Version 4	97.6	1243	3.1	24860	1868	0	0	0
Transmission Control Protocol	93.3	1189	94.2	763423	57 k	1188	763257	57 k
Hypertext Transfer Protocol	0.1	1	0.0	134	10	1	134	10
Open Shortest Path First	4.2	54	0.3	2376	178	54	2376	178
Address Resolution Protocol	0.5	6	0.0	168	12	6	168	12

Figura 1.15: Exemplo da Pilha Protocolar

Escalabilidade: Aquilo que observamos é que tanto o número de fluxos como o número de bps aumentam linearmente com as utilizações, logo não se trata de uma solução viável com escalabilidade.

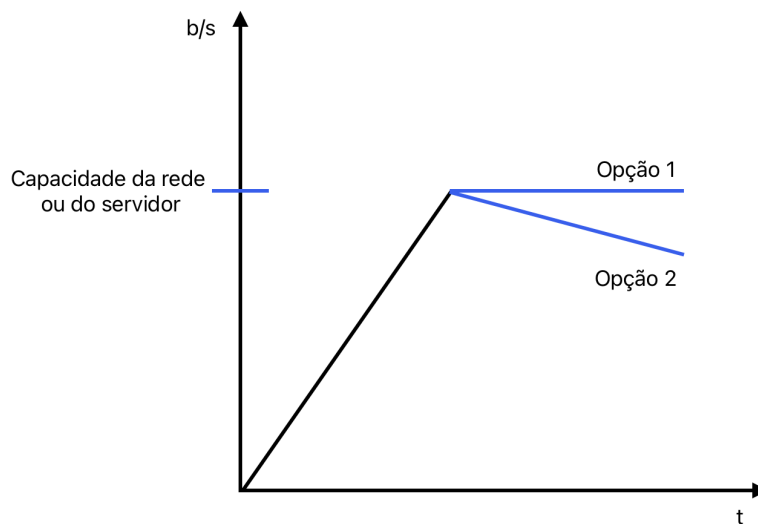


Figura 1.16: Assumindo um aumento de utilização linear ao longo de um tempo t

O que vemos é que existem duas hipóteses:

1. Manter a largura de banda quando o servidor ou a rede atingem o seu limite.

-
2. Perda de largura de banda utilizada no caso acima mencionado, nesta hipótese leva-se em conta os *timeouts* que os *requests* ao servidor podem ter, visto que se um fluxo usasse 10kb/s, numa situação limite acabaria por lhe ser "atribuído" menos, aumentando o tempo, fazendo com que o erro pudesse aparecer.

Capítulo 2

Etapa 2 - Streaming adaptativo sobre HTTP (MPEG-DASH)

2.1 Procedimento

Primeiramente, foram criados os vídeos com baixa resolução, conforme especificado no enunciado. Além disso, o arquivo de manifesto foi gerado. Foi, também, criado o arquivo *video_dash.html* para a visualização do vídeo e os arquivos JavaScript da biblioteca do DASH foram baixados.

```
core@xubuncore:~$ ls
core                Pictures                videoB_640_480_1000k_dash.mp4
dash.all.debug.js   Public                videoB_640_480_1000k.mp4
dash.all.min.js     Templates            videoB.mp4
Desktop            videoA.mp4           video_dash.html
Documents          videoB_200_150_200k_dash.mp4  video.html
Downloads          videoB_200_150_200k.mp4    video_manifest_init.mp4
Music              videoB_480_360_500k_dash.mp4  video_manifest.mpd
ospf-mdr           videoB_480_360_500k.mp4    Videos
```

Figura 2.1: Vídeos com resoluções diferentes e ficheiro de manifesto

Finalmente, o VStreamer foi colocado a servir o conteúdo com um servidor HTTP, mais especificamente, o *mini_httpd*.

```
root@VStreamer:/tmp/pycore.33807/VStreamer.conf# su - core
core@VStreamer:~$ mini_httpd -p 9999 -d ./ -D
bind: Address already in use
```

Figura 2.2: VStreamer a servir conteúdo com o *mini_httpd*

2.2 Questões

Questão 2: Diga qual a largura de banda necessária, em bits por segundo, para que o cliente de streaming consiga receber o vídeo no firefox e qual a pilha protocolar usada neste cenário.

Sendo que nesta etapa existe a possibilidade de fazer streaming de três videos de diferentes tamanhos,

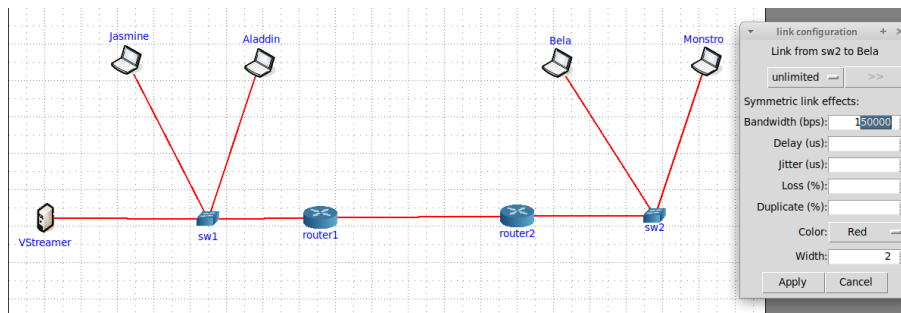


Figura 2.7: Streaming no cliente Aladdin

Após limitarmos a largura de banda nesta conexão, ocorreu a situação em que o cliente no laptop chamado *Bela* foi capaz de reproduzir o vídeo numa resolução mais baixa, ao passo que o cliente no laptop *Aladdin* continuou a exibir o vídeo em alta resolução. Conforme ilustrado na Figura 2.8, o cliente *Aladdin* com o endereço IP 10.0.0.21 reproduziu o vídeo em alta resolução, enquanto que o cliente *Bela*, identificado pelo endereço IP 10.0.2.21, conseguiu adaptar-se à situação. Inicialmente, o cliente *Bela* tentou transmitir o vídeo em alta resolução, alterando de seguida para uma resolução intermediária e, por fim, conseguiu transmitir o vídeo em menor resolução, conforme desejado. Embora não tenhamos conseguido visualizar o vídeo no navegador Firefox, possuímos a captura do Wireshark que comprova todo esse processo adaptativo realizado pelo cliente *Bela*, como podemos ver na figura 2.9.

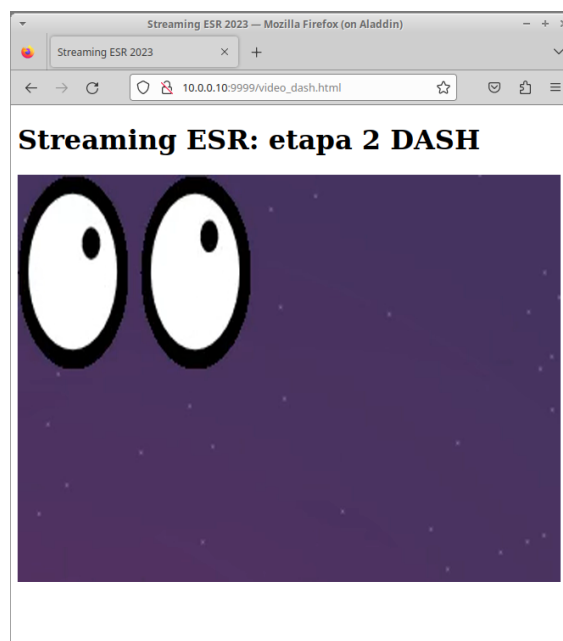


Figura 2.8: Streaming no cliente Aladdin

8952	326.096857115	10.0.2.21	10.0.0.10	HTTP	477	GET /video_dash.html HTTP/1.1
8954	326.114918156	10.0.0.10	10.0.2.21	HTTP	272	HTTP/1.1 304 Not Modified
8961	326.565049855	10.0.2.21	10.0.0.10	HTTP	411	GET /videoB_640_480_1000k_dash.mp4 HTTP/1.1
9059	330.211355242	10.0.2.21	10.0.0.10	HTTP	412	GET /videoB_480_360_500k_dash.mp4 HTTP/1.1
9135	332.854940366	10.0.2.21	10.0.0.10	HTTP	411	GET /videoB_200_150_200k_dash.mp4 HTTP/1.1
10128	360.567456832	10.0.2.21	10.0.0.10	HTTP	427	GET /video_dash.html HTTP/1.1
10131	360.571525692	10.0.2.21	10.0.0.10	HTTP	410	GET /videoB_200_150_200k_dash.mp4 HTTP/1.1
10134	360.575345395	10.0.2.21	10.0.0.10	HTTP	409	GET /videoB_200_150_200k_dash.mp4 HTTP/1.1
10137	360.579233876	10.0.2.21	10.0.0.10	HTTP	409	GET /videoB_200_150_200k_dash.mp4 HTTP/1.1
10140	360.584245788	10.0.2.21	10.0.0.10	HTTP	411	GET /videoB_200_150_200k_dash.mp4 HTTP/1.1
10143	360.588287621	10.0.2.21	10.0.0.10	HTTP	411	GET /videoB_200_150_200k_dash.mp4 HTTP/1.1
10165	360.921417451	10.0.2.21	10.0.0.10	HTTP	410	GET /videoB_200_150_200k_dash.mp4 HTTP/1.1
10166	360.922235917	10.0.2.21	10.0.0.10	HTTP	411	GET /videoB_200_150_200k_dash.mp4 HTTP/1.1

Figura 2.9: Captura do Wireshark do streaming no cliente Bela com limitação da largura de banda

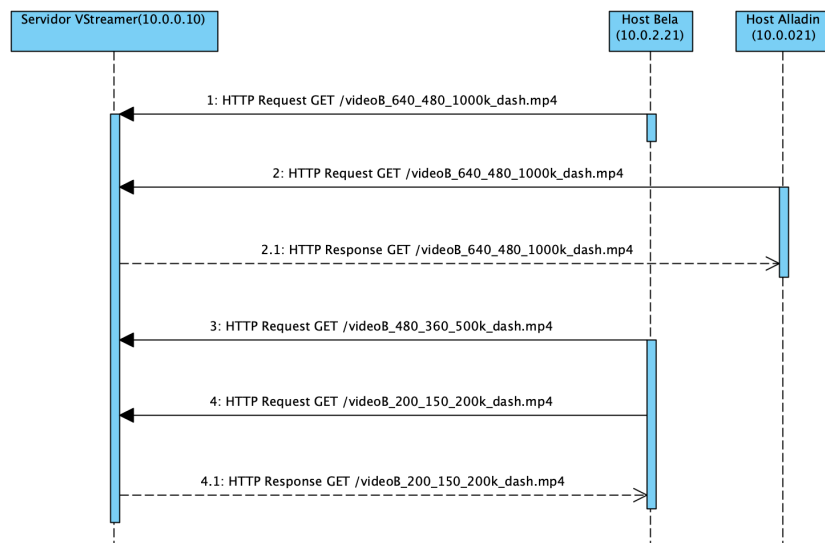


Figura 2.10: Trama de streaming com débito adaptativo

Questão 4: Descreva o funcionamento do DASH neste caso concreto, referindo o papel do ficheiro MPD criado.

O DASH possui uma função que permite ajustar automaticamente a qualidade do vídeo de acordo com a capacidade da rede, o que resulta numa experiência melhor para o utilizador. O arquivo MPD, que está no formato XML, elenca todas as opções de resolução disponíveis para o vídeo que será transmitido. Neste caso, temos três opções diferentes. A transmissão é dividida em segmentos de 500 milissegundos (usando o parâmetro *-dash 500*), e a resolução do vídeo pode mudar entre esses segmentos. Assim sendo, a qualidade da transmissão pode variar, isto é, por exemplo, um segmento pode ser transmitido em baixa resolução e o próximo em alta resolução.

Capítulo 3

Etapa 3 - Streaming RTP/RTCP unicast sobre UDP e multicast com anúncios SAP

3.1 Topologia *Unicast*

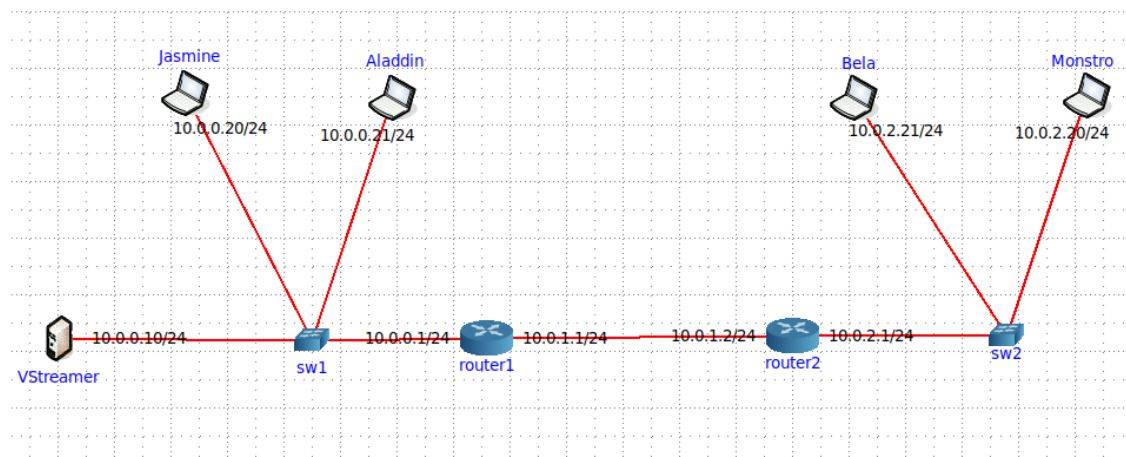


Figura 3.1: Tipologia de Rede para o exercício *Unicast*

3.2 Procedimento *Unicast*

O procedimento utilizado foi exatamente o requisitado:

- No servidor *VStreamer* iniciamos uma sessão de *streaming* com RTP com o *ffmpeg*.
- Verificamos a conectividade com o portátil **Monstro** antes de iniciar um cliente no mesmo.

```
root@VStreamer:/tmp/pycore.34783/VStreamer.conf# ping 10.0.2.20
PING 10.0.2.20 (10.0.2.20) 56(84) bytes of data.
64 bytes from 10.0.2.20: icmp_seq=1 ttl=62 time=0.095 ms
64 bytes from 10.0.2.20: icmp_seq=2 ttl=62 time=0.081 ms
64 bytes from 10.0.2.20: icmp_seq=3 ttl=62 time=0.089 ms
^Z
[1]+  Stopped                  ping 10.0.2.20
root@VStreamer:/tmp/pycore.34783/VStreamer.conf#
```

Figura 3.2: Ping teste do VStreamer para o portátil **Monstro**

- Iniciamos um cliente *ffplay* no cliente **Monstro**.
- Capturamos o tráfego no link de saída do servidor com o Wireshark.

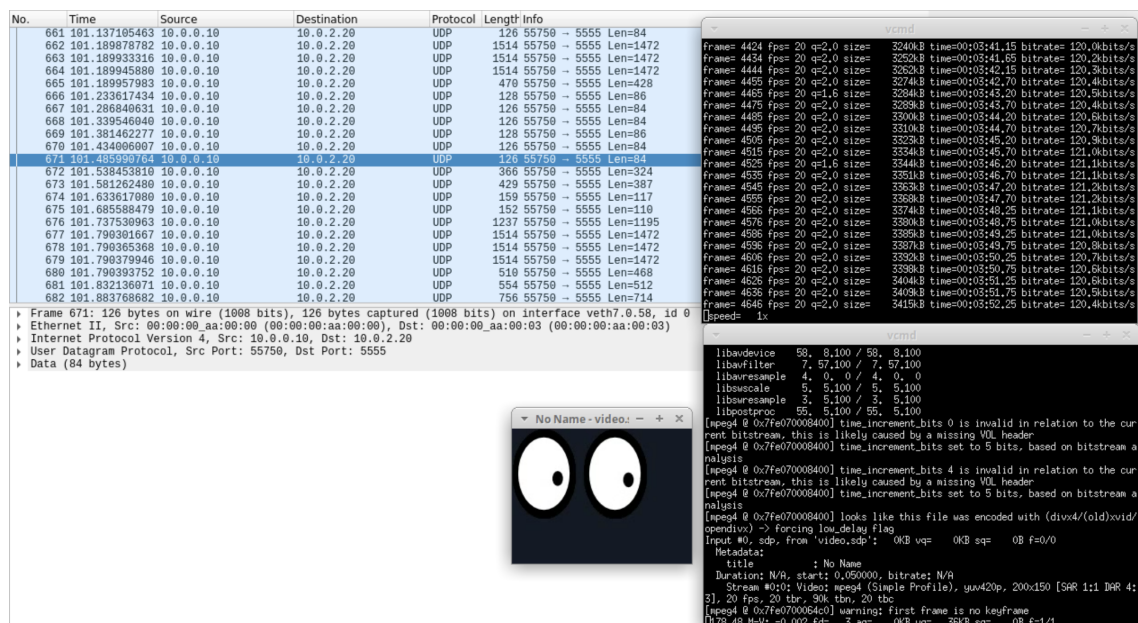


Figura 3.3: VStreamer a transmitir para o portátil **Monstro**

3.3 Topologia *Multicast*

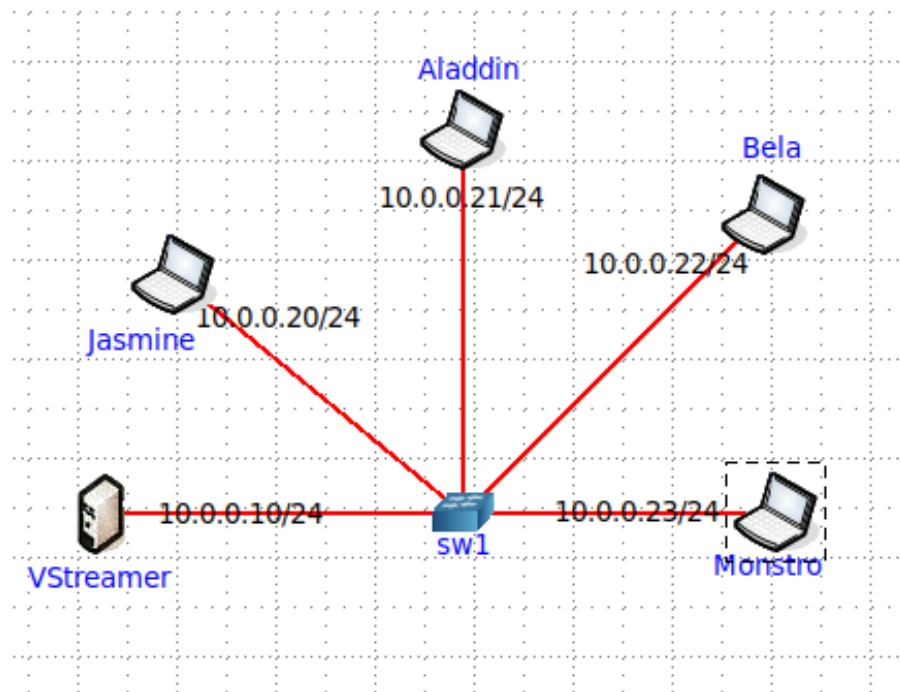


Figura 3.4: Tipologia de Rede para o exercício *Multicast*

3.4 Procedimento Multicast

- No servidor VStreamer iniciamos uma sessão de *streaming multicast* com o *ffmpeg*.
- Verificamos a conectividade com todos os portáteis antes de iniciar um cliente nos mesmos.

```
root@VStreamer:/tmp/pycore.41643/VStreamer.conf# ping 10.0.0.20
PING 10.0.0.20 (10.0.0.20) 56(84) bytes of data.
64 bytes from 10.0.0.20: icmp_seq=1 ttl=64 time=0.031 ms
64 bytes from 10.0.0.20: icmp_seq=2 ttl=64 time=0.048 ms
64 bytes from 10.0.0.20: icmp_seq=3 ttl=64 time=0.080 ms
^Z
[1]+  Stopped                  ping 10.0.0.20
root@VStreamer:/tmp/pycore.41643/VStreamer.conf# ping 10.0.0.21
PING 10.0.0.21 (10.0.0.21) 56(84) bytes of data.
64 bytes from 10.0.0.21: icmp_seq=1 ttl=64 time=0.045 ms
64 bytes from 10.0.0.21: icmp_seq=2 ttl=64 time=0.071 ms
64 bytes from 10.0.0.21: icmp_seq=3 ttl=64 time=0.056 ms
^Z
[2]+  Stopped                  ping 10.0.0.21
root@VStreamer:/tmp/pycore.41643/VStreamer.conf# ping 10.0.0.22
PING 10.0.0.22 (10.0.0.22) 56(84) bytes of data.
64 bytes from 10.0.0.22: icmp_seq=1 ttl=64 time=0.034 ms
64 bytes from 10.0.0.22: icmp_seq=2 ttl=64 time=0.050 ms
64 bytes from 10.0.0.22: icmp_seq=3 ttl=64 time=0.075 ms
^Z
[3]+  Stopped                  ping 10.0.0.22
root@VStreamer:/tmp/pycore.41643/VStreamer.conf# ping 10.0.0.23
PING 10.0.0.23 (10.0.0.23) 56(84) bytes of data.
64 bytes from 10.0.0.23: icmp_seq=1 ttl=64 time=0.034 ms
64 bytes from 10.0.0.23: icmp_seq=2 ttl=64 time=0.072 ms
64 bytes from 10.0.0.23: icmp_seq=3 ttl=64 time=0.071 ms
^Z
[4]+  Stopped                  ping 10.0.0.23
root@VStreamer:/tmp/pycore.41643/VStreamer.conf#
```

Figura 3.5: Ping teste do VStreamer para todos os portáteis da topologia

- Em cada um dos portáteis (**Jasmine**, **Aladdin**, **Bela** e **Monstro**) iniciamos um cliente da sessão diferente.
- Visualizamos a stream em todos os portáteis.

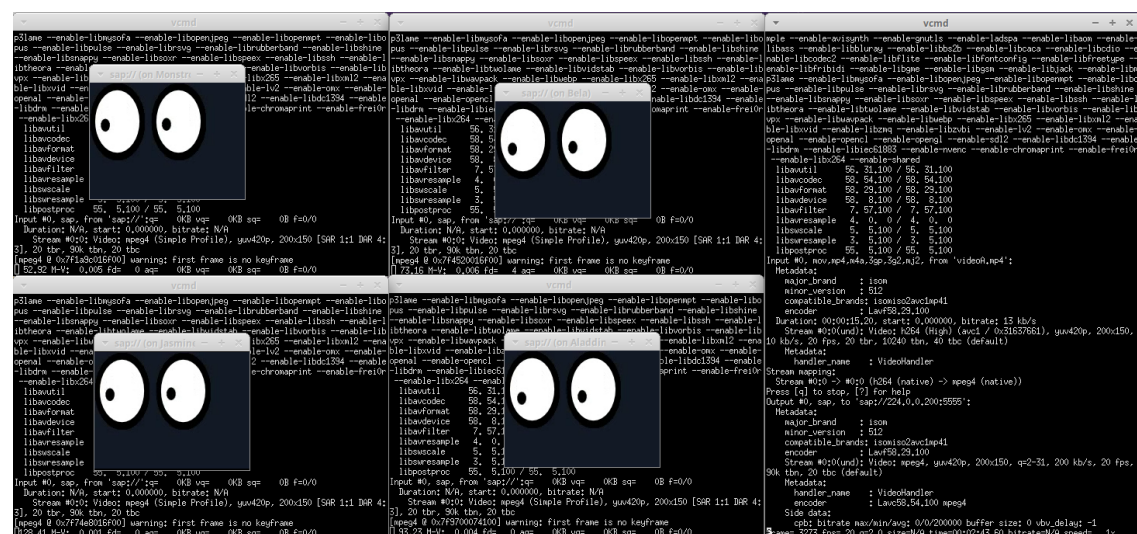


Figura 3.6: VStreamer a transmitir para todos os portáteis da rede

3.5 Questões

Questão 5: Compare o cenário unicast aplicado com o cenário multicast. Mostre vantagens e desvantagens na solução multicast ao nível da rede, no que diz respeito a escalabilidade (aumento do nº de clientes) e tráfego na rede. Tire as suas conclusões.

A fácil configuração do cenário *unicast* é a única vantagem a apontar. Neste cenário, o servidor centralizado aloca um canal de transmissão de dados para atender aos pedidos de cada cliente. Isto faz com que a banda larga necessária para atender a vários pedidos seja diretamente proporcional à taxa de chegada dos mesmos. Sendo que a banda larga é um recurso limitado, isto afeta a solução *unicast* a nível de escalabilidade.

Por outro lado, no cenário *multicast* a transmissão de dados é feita para vários clientes simultaneamente. Este serviço é mais escalável, já que o aumento do número de clientes não afeta tanto a transmissão. Além disso, o uso da largura de banda é mais eficiente. Contudo, este cenário também apresenta algumas desvantagens. Ao contrário do serviço *unicast*, aquele primeiro não possui nenhum mecanismo de correção de erros, logo, para tornar a solução confiável, esses mecanismos deverão ser implementados na camada de aplicação.

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s
Frame	100.0	2393	100.0	1441530	69 k	0	0	0
Ethernet	100.0	2393	2.3	33502	1605	0	0	0
Internet Protocol Version 6	1.0	25	0.1	1000	47	0	0	0
Internet Protocol Version 4	98.6	2360	3.3	47200	2261	0	0	0
User Datagram Protocol	92.9	2222	1.2	17776	851	0	0	0
Data	91.7	2195	91.4	1317189	63 k	2195	1317189	63 k

Figura 3.7: Cenário *Unicast*

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s
Frame	100.0	9367	100.0	5932762	129 k	0	0	0
Ethernet	100.0	9367	2.2	131138	2859	0	0	0
Internet Protocol Version 6	0.2	15	0.0	600	13	0	0	0
Internet Protocol Version 4	99.8	9352	3.2	187100	4079	0	0	0
User Datagram Protocol	99.7	9337	1.3	74696	1628	0	0	0
Session Announcement Protocol	0.8	73	0.4	23652	515	0	0	0
Real-Time Transport Protocol	98.1	9191	92.9	5512896	120 k	0	0	0
MP4V-ES	98.1	9191	91.1	5402604	117 k	9191	5402604	117 k
Real-time Transport Control Protocol	0.8	73	0.0	2044	44	73	2044	44
Internet Group Management Protocol	0.2	15	0.0	248	5	15	248	5

Figura 3.8: Cenário *Multicast*